INDEX

S.NO	TOPICS	PAGE NO
1	**ABSTRACT**	1
2	**INTRODUCTION**	3
	2.1 DEEP LEARNING FOR MALARIA DIAGNOSIS	5
3	**LITERATURE REVIEW**	6
	3.1 MALARIA AND ITS DIAGNOSIS	8
	3.2 MACHINE LEARNING APPLICATIONS IN HEALTH CARE	9
	3.3 DEEP LEARNING APPLICATIONS IN HEALTH CARE	12
4	**SYSTEM ANALYSIS**	13-15
5	**SYSTEM SPECIFICATION**	16
	5.1 DATA SET	17
	5.2 HARDWARE	17
	5.3 TECHNIQUES	18
6	**DETAILS DESCRIPTION OF TECHNOLOGY**	19
	6.1 SETTING UP THE CLOUD	20
	6.2 PREPROCESSING	21
	6.2.1 LABLE ENCODING	22
	6.2.2 NORMALIZATION	23
	6.2.3 STANDARISATION	24

	6.2.4 DATA AGUMENTATION	25
	6.3 MODEL ARCHITECTURE AND IMPLEMENTATION	25
	6.3.1 TRANSFER LEARNING	26
	6.3.2 VGG-19	27
	6.3.3 RESNET-50	28
	6.3.4 ENSEMBLE MODEL	29
	6.4 COFUSION MATRIX ANAYSIS	31
	6.5 PERFORMANCE PARAMETER	32
7	**SYSTEM TESTING**	33
	7.1 TYPES OF TESTING	38
8	**IMPLEMEMTATION**	39
	CODE FOR EXTRACTING THE DATA	41
	PREPROCESSING STEPS	46
	FROZEN PRE-TRAINED CNN	48
	CODE FOR EXTRACTING THE DATA	52
9	**RESULT**	58
10	**DISCUSSION**	61
11	**SCREEN SHOT**	63
12	**CONCLUSION**	65
13	**BIBILOGRAPHY**	72

ABSTRACT

ABSTRACT

Malaria is a female Anopheles mosquito-borne disease that transmits a motile infec- tive form to the host body such as humans, which reproduce asexually in the blood cells of the host. The typical symptoms of malaria are fever, headache, tiredness, and vomiting. In severe cases may lead to coma and death. In this research, we useddeep neural networks to detect the malaria virus in human blood cells. Traditional malaria detection techniques require experts to test blood cells under a microscope.The shortage of skilled technicians and the unavailability of required equipment and infrastructure resultin false diagnoses leading to an increase in mortality rate. Early researches have shown detection using Machine Learning techniques like Support VectorMachine (SVM), whichis tedious and requires hand-engineered feature extraction to train data, and the results were not up to the mark. The proposed method in this re- search shows a system with end-to-end automated models using a deep neural network that performs both feature extraction and classification using blood smear cell images. The dataset used in this research was taken from the National Institute of Health (NIH) Malaria Dataset. Modelsare evaluated based on accuracy, precision, recall, and F1- score. Data Preprocessing techniques like data segmentation and normalization are applied to maximize the modelperformance. In this research, two deep Convolutional Neural Networks, VGG-19, and ResNet-50, are experimented on to analyze and com- pare the best performing model onthe malaria dataset. Our results showed that the ResNet-50 outperforms the VGG-19 byachieving an accuracy of 97 percent.

INTRODUCTION

2. INTRODUCTION

Malaria is a mosquito-borne disease caused by a plasmodium parasite transmitted by the bite of infected mosquitoes. Worldwide, an estimated 300–500 million people con- tract malaria each year, resulting in 1.5–2.7 million deaths annually [2]. Accordingto the World Health Organization (WHO), approximately 219 million cases were di- agnosed with malaria resulting in 435,000 deaths globally in 2017 [3]. Malaria is a deadly disease which is more frequently found in rural areas where medical diagnosis and health care options are not easily accessible.

Worldwide accepted light microscopy technique is used by the practitioners for the diagnosis of malaria. The conventional light microscopy for malaria diagnosis uses thick and thin stained blood smears for diagnoses. Microscopy is well adapted in areas which are highly prone to malaria. The major drawback of this approach is its dependence on skilled technicians, of which there is a critical shortage. A nationwide study in Ghana found 1.72 microscopes per 100,000 population, but only .85 trained laboratory staff per 100,000 population [4]. This results in a delay in an inaccurate diagnosis. Most of the time, diagnosis is often made based on clinical signs and symptoms alone, whichis error-prone and this inaccurate diagnosis leads to higher mortality, drug resistance, and the economic burden of buying unnecessary drugs.

Early and accurate malaria diagnosis and prompt treatment can cure a patient and save many lives by preventing severe malaria cases. All around the world, there are

millions of people who are still lacking access to malaria prevention and treatment. Therefore, there is a need for a reliable alternative which will help to provide access to high-quality diagnosis routinely that is currently unavailable.

This research focuses on designing an accurate malaria diagnosis model that canbe implemented without any dependencies on skilled technicians and testing the model accuracy to get high-quality results. Automated image analysis software could remove the most serious limitation of the worldwide accepted microscopy method in general, dependency on human experts for diagnostic accuracy of the results. Automating the detection process means using the knowledge, the practice of conventional methods, and implementing it to get fast and efficient results.

2.1 Deep Learning for Malaria diagnosis

Machine learning is a field of science where decisions are made by analyzing data and reading the data to get the desired output. In the past decade, the health sector has seen a significant growth, a lot of research is ongoing to make healthcare automated to make the process easily available to everyone.

To automate the diagnosis process, much researches have been conducted on various machine learning models. Machine learning models require a lot of tuning, factor analysis, and feature engineering. The machine learning method is not scalable with more data provided. Machine learning requires feature engineering and feature training which is not a handy tool. Deep learning models are reliable and easily scalable with a higher accuracy rate. Convolutional Neural Networks (CNN) have proved to be really effective in a wide variety of computer vision tasks [4]. Convolutional Neural Networks transforms input image volume into an output volume holding a class label. In Convolutional modeling, there is no need for feature extraction and feature engineering as extracting the features with the most variance like which feature holds more impor-

tance. CNN models take the images as an input, the image is passed on different layers of the CNN model which is discussed in further sections. This makes Convolutional Neural Network an optimal approach for diagnosing Malaria using patient blood cell images.

In this research, we attempted to answer two research questions:

1. Compare models to find which Convolutional Neural Network between RESNET-50 and VGG-19 performs better on the malaria dataset?

2. Is there any difference in the Convolutional Neural Network performance measured on the malaria dataset before and after applying preprocessing techniques like data augmentation, stain normalization and standardization

LITERATURE REVIEW

3.LITERATURE REVIEW

3.1 Malaria and Its Diagnosis

Malaria is a disease of the blood that is caused by the Plasmodium parasite, which is transmitted from person to person by a particular type of mosquito. Malaria isone of the world's deadliest diseases and remains one of the top child killers on the planet[2]. Delay in diagnosis and treatment, malaria infection can develop to anemia, hypoglycemia, or cerebral malaria, in which capillaries carrying blood to the brain are blocked. Cerebral malaria can cause coma, life-long-learning disabilities, and death [5]. Early diagnosis of malaria can reduce mortality. Widely accepted microscopic diagnosis is the most popular malaria diagnosis method where malaria is diagnosed microscopically by staining thick and thin blood films on a glass slide, to visualize malaria parasites. The prepared slides are visualized under a microscope by expert technicians, the wide acceptance of this method can be attributed to its simplicity and its ability to identify the presence of parasites. However, the process is labor-intensive, time-consuming, and requires expert and trained healthcare workers. The shortage and unavailability of skilled technicians and health facility resources in remote rural areas make the microscopic diagnosis inefficient. Most people in rural areas use some sort

of self-treatment based on symptoms which result in a false diagnosis and improper treatment. Studies from endemic countries such as India and South Africa have also shown that laboratory misdiagnosis is not uncommon [6]. Misdiagnosis of malaria can be due to two major reasons, immense workload and limited human resources with the primary reason being the shortage of skilled technician.

Fifteen countries in sub-Saharan Africa and India carried almost 80 percent of the global malaria burden [3]. In the countries with high malaria cases, it is evident that there is a high requirement of skilled technicians, as per the survey conducted in Ghana,there is less than two technician for 100,000 cases, which results in inaccurate readingsdue to burden on the technician [7]. Other countries where malaria is not that common do not have skilled technicians; the technicians are not well trained as they lack to witness the malaria cases very often. Automating the detection process will help in easy and effective diagnosis all around the world. Data can be collected globally and the model can be trained on the global data set to give more accurate results.

The light microscopy is the gold standard technique for malaria diagnosis on Giemsa-stained thin and thick blood films, which requires a high level of expertise. This is not always available around the clock outside specialized centers and Rapid Diagnostic Tests (RDTs) are, therefore, increasingly used. However, RDTs have limitations and once a malaria diagnosis has been made, the infecting species present must be deter- mined. The primary method of malaria diagnosis microscopy its evaluation accuracy mostly depends on the expert's clinic-pathological understanding. In effect, such a pro-cedure involves a humanistic error in terms of subjectivity, which leads to inconsistent as well as less diagnostic accuracy. To increase diagnostic precision by minimizing such subjectivity, developing a computer-assisted malaria parasite detection tool has given importance in modern pathological services where a clinician will get assistance to make a better decision toward malaria diagnosis. In modern diagnostic system development, machine learning techniques have enormous contributions for achieving higher diag-

nostic precision in medical imaging informatics like microscopy, ultrasound imaging, MRI, and CT [8]. Microscopic image analysis is the most important as well as a highly informative tool toward a pathological evaluation of different diseases, hematological disorder, oral cancer, breast cancer, cervical cancer, etc. Like other microscopic image analysis, peripheral blood smear screening is one of the essential diagnostic techniques to identify hematological disorders (anemia, thalassemia, etc.) and parasitic infection (malaria, filaria) in the blood. In the case of malaria detection, pathologists frequently use a light microscope to detect infection in erythrocytes based on color as well as morphological changes of the erythrocyte [9].

3.2 Machine Learning Applications in Health Care

Computer-aided diagnostic tools using machine learning (ML) algorithms applied to microscopic blood smear images have the potential to reduce clinical burden by assisting with triage and disease interpretation. These tools process medical images for typical appearances and highlight pathological features to supplement clinical decision-making. For these reasons, Computer-aided diagnostic tools have gained prominence in image-based medical diagnosis and risk assessment. Research by Das et al. [8] and Park etal. [10], Bayesian learning, support vector machines, logistic regression, and k-nearest neighbor algorithm perform well in this context. Also, preprocessing attempts like stain removal from peripheral blood smear images and impulse noise reduction were implemented. In research, Mustafa et al. [11] proposed thresholding as a preprocessing step, which is considered to be one of the most important preprocessing steps in this task. In their work, a comparison among the Fuzzy C-Mean algorithm [12], Wolf's method [13], Bradley's method [14] and Bernsen's method [15] has been shown. Each of these methods experiments with malaria images dataset. However, a majority of these tools applied to malaria diagnosis use handcrafted feature extraction algorithms

that are optimized for individual datasets and trained for specific variability in source machinery, dimension, position, and orientation of the region of interest (ROI) [8].

Many researchers have conducted their study on the malaria diagnosis and have achieved great results, but with the advancement in the technology and availability of much more data, the new field of Artificial Intelligence is gaining popularity thatis Deep Learning. Deep learning is an artificial intelligence function that imitatesthe workings of the human brain in processing data and creating patterns for use in decision making. Deep learning is a subset of machine learning in artificial intelligence(AI) that has networks capable of learning unsupervised from data that is unstructured or unlabeled. It is also known as deep neural learning or deep neural network [16]. Withthe advancement in technology, researchers have uncovered new ways that make data analysis and prediction more efficient and accurate compared to machine learning models. Prior research on Malaria dataset using machine learning was done on SVM (Support Vector Machines) in which the researchers had to preprocess the dataset to remove noises and preserve the important features, also process included image contrast enhancement and illumination correctness. The next steps of the process were segmentation, feature computation and selection, and then classification [17]. The machine learning model after extensive work and analysis was not as accurate as it was required to be to replace the manual analysis. Further research on deep convolutional networks resulted in exceptional results in different image datasets. The Alex Net neural network, the model that gained recognition in the ImageNet challenge showed exceptional results on the image recognition dataset.

3.3 Deep Learning Applications in Health Care

At present, data-driven deep learning (DL) methods have superseded the performance of handcrafted feature extraction mechanisms by self-discovering the attributes from

raw pixel data and performing end-to-end feature extraction and classification [18]. In particular, the convolutional neural networks (CNN), a class of DL models, have demonstrated promising results in image classification, recognition, and localization tasks [19]. The promising performance of CNNs is attributed to the availability of huge amounts of annotated data.

Under circumstances of limited data availability as in the case of medical images, transfer learning strategies are adopted. Var et al. [20] and [21] proposed methods for computer aided diagnosis based on pre-trained convolutional neural networks as feature extractors to identify malaria parasites. The CNN models are pretrained on large-scale datasets like ImageNet [22] to transfer the knowledge learned in the form ofgenericimage features to be applied for the target task. The pretrained weights serveas a good initialization and are found to perform better than training the model from scratch with randomly initialized weights.

Literature studies reveal the application of conventional ML and data-driven DL methods toward the challenge of malaria parasite detection in thin-blood smear im- ages. Dong et al. [23] compared the performance of kernel-based algorithms such as support vector machine (SVM) and CNN's toward classifying infected and normal cells.A small-scale collection of segmented red blood cells (RBCs) were randomly split into train/validation/test sets. It was observed that the CNNs achieved a classification ac- curacy of over 95% and significantly outperformed the SVM classifier that obtained 92% accuracy. The CNNs self-discovered the features from the raw pixel data, thereby requiring minimal human intervention for automated diagnosis. Liang et al. [24] per- formed cross-validation studies at the cell level to evaluate the performance of custom and pre-trained CNN models toward classifying parasitized and normal cell images.

Experimental results demonstrated that the custom CNN outperformed the pre-trained AlexNet [19] model with an accuracy of 97.37%. In another study [25], the authors performed randomized splits with peripheral smear images and evaluated the

performance of a shallow deep belief network toward detecting the parasites. Experimental results demonstrated that the deep belief network showed promising performance with an F-score of 89.66% as compared to that of SVM based classification that gave an F-score of 78.44%. A CNN model was customized to analyze focal stack of slide images for the presence of parasites [26]. In the process, they observed that the custom CNN model achieved a Matthews Correlation Coefficient (MCC) score of 98.77% and considerably outperformed the SVM classifier that achieved 91.81% MCC. These studies were evaluated at the cell level, with randomized splits and/or small-scale datasets. The reported outcomes are promising; however, patient-level cross-validation studies with large-scale clinical datasets are required to substantiate their robustness and generalization to real-world applications. Rajaraman [21] used a large-scale, annotated clinical image dataset, extracted the features from the optimal layers of pre-trained CNNs and, statistically validated their performance at both cell and patient-level to- ward discriminating parasitized and uninfected cells.

SYSTEM ANALYSIS

4.SYSTEM ANALYSIS

The purpose of this study is to compare and contrast the effectiveness of deep learning models, specifically Convolutional Neural Networks, in categorizing human blood smear images as Malaria Parasitized vs. Healthy. This research also focuses on uncovering the effects of preprocessing techniques on model accuracy.

Many researches have been conducted on automating the malaria diagnosis using machine learning [27], [17] and [10]. Machine learning has shown great results in the health care industry like the google's machine learning algorithm to help identify cancerous tumors on mammograms [28]. Research on diagnosing diabetic retinopathy in retinal images is one of the most successful discovery in the IT and Health sector [29]. Many researchers have conducted their study on the malaria diagnosis and have achieved great results, but with the advancement in the technology and availability of much more data, new field of Artificial Intelligence is gaining popularity that is Deep Learning. Deep learning is a sub field of machine learning that imitates the workings ofthe human brain in processing data and creating patterns for use in decision making. In this research, we compared the results gained during the research with the preexisting results using machine learning techniques in earlier studies

Research on CNNs were not focused on preprocessing techniques, rather they were

more relied on getting comparable results using CNNs and transfer learning. Most researches were limited to designing CNN from scratch. We have conducted extensive experiments using the NIH malaria dataset [30] on three different settings, namely, custom network from scratch, fine tuning on pre-trained model and Ensemble model

SYSTEM SPECIFICATION

SYSTEM SPECIFICTION

5.1 Dataset ：

The dataset is taken from the National Institute of Health's (NIH) official website. The dataset contains 27,557 images: 13,778 infected images, and 13,779 non-infected images. The dataset is collected by researchers at Lister Hill National Center for Biomedical Communications (LHNCBC) part of the National Library of Medicine. Giemsa-stained thin blood smear slides from 150 P.falciparum-infected and 50 healthy patients were collected and photographed [31]. The images were manually labeled by an expert slide reader at the Mahidol-Oxford Tropical Medicine Research Unit in Bangkok, Thailand.

5.2 Hardware

A virtual machine with 16 central processing units (CPUs),

Google Cloud Platform: Google Cloud Platform is a set of Computing, Networking, Storage, Big Data, Machine Learning and Management services provided by Google that runs on the cloud infrastructure that Google uses internally for its end-user products

5.3 Techniques:

Deep Learning

Keras

Tensorflow

We used Python script for this project. As a framework, we used Keras 2.3, which is a high-level neural network API written in Python. But Keras can not work by itself; it needs a backend for low-level operations. Thus, we installed a dedicated software library — Google's TensorFlow 2.2.

CNN Models: In Deep Learning, Convolutional Neural Network (CNN) is a type of an Artificial Neural Network. CNN or ConvNet is a class of deep, feed-forward artificial neural systems, most normally connected to examining visual representations. CNN is widely used for image recognition, images classifications, objects detections, recognition faces, etc. [33]. In this research, we used two different CNN models: ResNet-50 and, VGG-19.

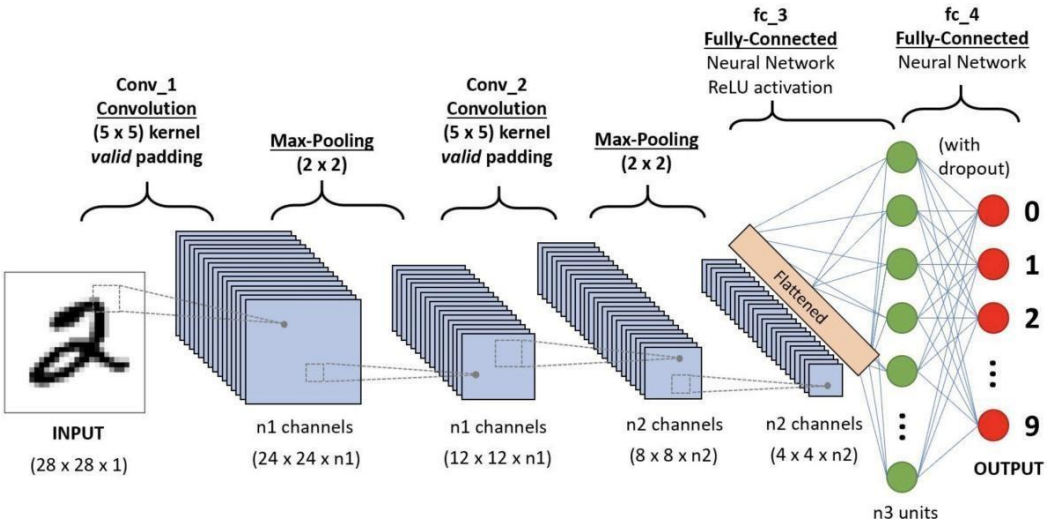


Figure 5.1: CNN Workflow [1]

DETAIL DESCRIPTION OF TECHNOLOGY

6. DETAIL DESCRIPTION OF TECHNOLOGY

6.1 Setting Up the Cloud

This research focuses on comparing performances of different deep Convolutional Neural Networks which requires high computational power for execution. The Graphics Processing Unit (GPU) or multiple CPU computation is needed to run deep neural networks. In this research, we used google cloud services to get a virtual machine with 16 CPUs to perform operations on the malaria dataset.

6.2 Preprocessing

This section will help understand all the preprocessing techniques used in the research to record the model performance difference. The malaria dataset consists of 27,557 images of which 13,778 are parasitized images and 13,779 are non-infected images. We merged files of both non-infected images and parasitized images and labeled them as healthy and malaria for the respective files. The dataset is divided into three sets for training, validation, and testing.

The ratio of train to validation to test is 63:7:30. We choose to take only 63% of dataset as training to avoid high computational intensity due to large dataset. The

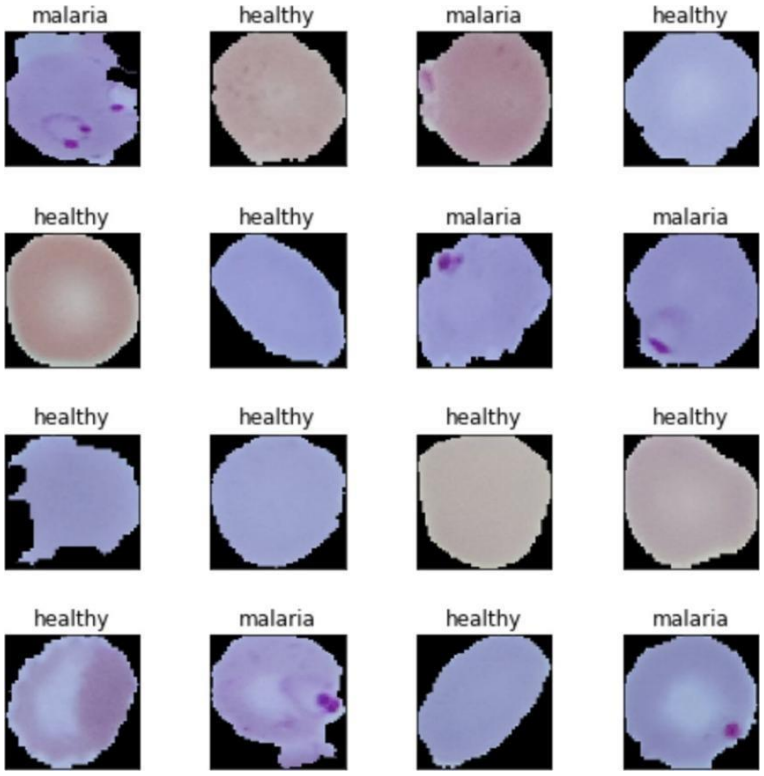


Figure 6.1: Malaria Dataset

training dataset consists of 63% of the dataset with 8707 healthy images and 8653 malaria images, the validation dataset consists of 7% with 1001 healthy and 928 malaria images, and the testing dataset is 30% of the whole dataset having 4144 and 4124 healthy and malaria images, respectively.

After analyzing the dataset, images in the dataset have different dimensions varying from [46, 58] to [385, 394] over the dataset. Scaling images up gives an advantage in performance but also requires a lot of computation time and memory space. It is betterto maintain this trade-off of accuracy and computation. In this research, we decidedto resize all the images to [125,125]. This resulted in a better classification score withgood processing speed.

Since neural networks receive inputs of the same size, all images need to be resized to a fixed size before inputting them to the CNN [23]. The larger the fixed size,

the less shrinking required. Less shrinking means less deformation of features and patterns inside the image. This will mitigate the classification accuracy degradation due to deformations. However, large images not only occupy more space in the memorybut also result in a larger neural network. Thus, increasing both the space and time complexity. It is obvious now that choosing this fixed size for images is a matter of trade-off between computational efficiency and accuracy.

6.2.1 Label encoding

Label Encoding is a technique of converting categorical variables to a numeric value. In figure 5.2 each value in the column is converted to a number. In this research, we used label encoding to encode the target dataset. The labels Malaria and healthy are encoded as 1 and 0, respectively. The label encoder is implemented only on the trainingand validation datasets.

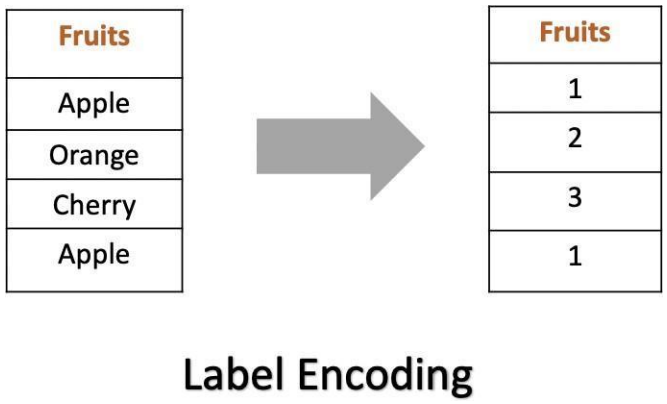


Figure 6.2: Label Encoding

6.2.2 Normalization

Normalization is an important preprocessing task which reduces the color and intensity variations present in stained images from different laboratories. According to past

research, stain normalization has proven to significantly increase the accuracy of the unseen dataset by approximately eight percent [34]. In this research, the images are collected from the preexisting dataset of human blood cells which is prepared from in laboratory examination. The smear slides are prepared in the laboratory using various chemical stains which results in color variation due to the use of different chemicals and staining procedures. This staining results in the model to learn and deal with more complex models with a diverse set of images leads to maximizing error rate. A solution to standardize this is normalization.

Stain Normalization is a common preprocessing technique that attempts to reduce color variability and improve the generalization of algorithms by transforming the input data to a common space. In stain normalized digital pathology samples, regions of digital tissue specimens are mapped to similar color characteristics regardless of the scanning device, stain vendor, and preparation protocols. Because of the reduced variability in color characteristics of tissues, Stain Normalization has demonstrated improvement in computer-assisted diagnostic tools [35, 36, 37].

In their article, Cimopi et al. [31] have shown how methods like histopathology [38] and stain normalization can improve the classification of colorectal tissues in colorectal cancer [31]. In this research, we have implemented stain normalization on the training and validation dataset while leaving the test dataset untouched.

After implementation as shown in plots that range of images is transformed into a narrower range and it is evident that the semantic meaning of the images is preserved as shown in the Figure 5.6.

6.2.3 Standardization

Standardization is a preprocessing technique that helps the dataset in two ways; first, it puts the data on the same scale, and second, balancing the data points around a point can be called centering. In this research, as we are explicitly specifying the dimensions

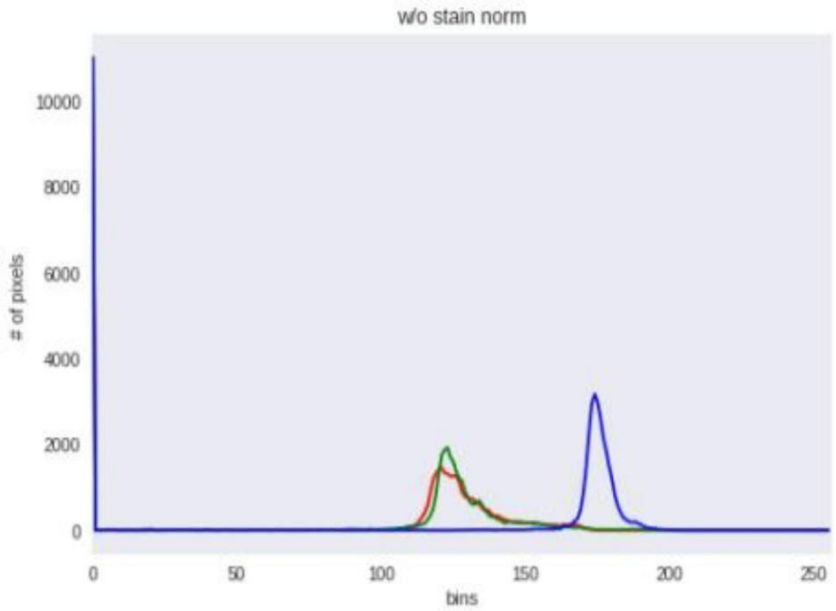


Figure 6.3: Images range without normalization

of the image the first point doesn't make much sense but concerning centering data points it is discussed in many pieces of research that Centering fights to vanish and ex- ploding gradients, while probably also increasing convergence speed and accuracy [39].After implementing the preprocessing technique strain standardization, standardization experiments where each feature value is rescaled to have mean μ=0 and standard deviation σ=1. The standardization method is implemented on both training and validation datasets.

6.2.4 Data Augmentation

Data Augmentation is a technique that is applied to the dataset to increase the variety of data available for training models. It has been used in the various medical dataset to improve classification performance [40],[41]. Majorly the dataset with small size uses data augmentation to increase the number of samples to improve the model performance by supplying variable data values for the model to train on. The dataset consists of equal instances in different classes keeping it a balanced dataset. Data augmentation

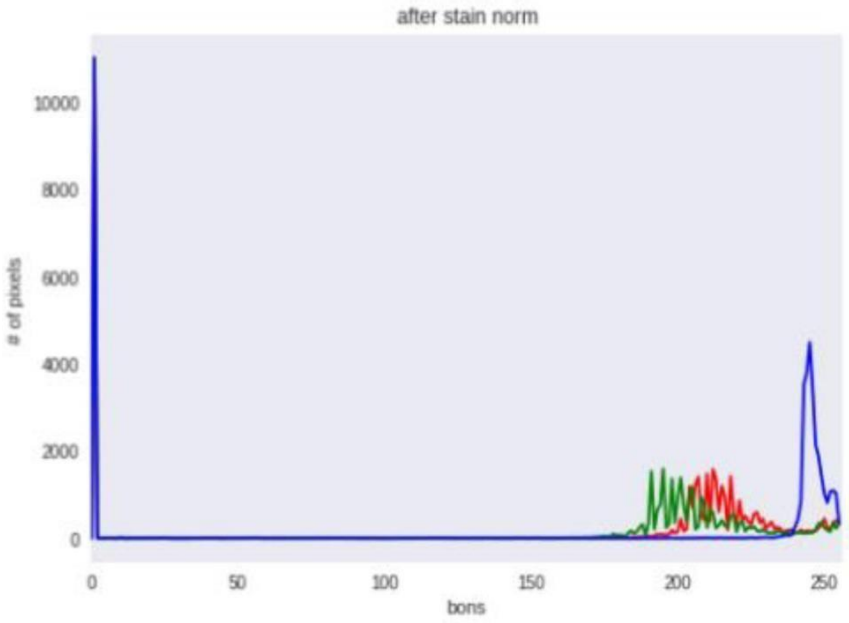


Figure 5.4: Images range with normalization

is performed on the training dataset which eliminates the chances of having similarity with the test dataset.

Data augmentation applied includes rotation, flip, horizontal and vertical shifts, lightening the images, darkening, zoom in, and zoom out. All these methods are applied in a random fashion which increased the training set from 27,678 to 140,123 data points.

6.3 Model Architecture and Implementation

6.3.1 Transfer Learning

Transfer Learning is a feature that enables users to transfer the knowledge of pre-trained models and use it in their own problem set. Instead of creating a model from scratch in this research, we used the models that are trained on large datasets such as ImageNet with 100,000 datapoints and explored the power of transfer learning which is proven to be significant in many image classification types of research [42]. In this research, we recognized that CNNs trained on large-scale datasets could serve as feature extractors

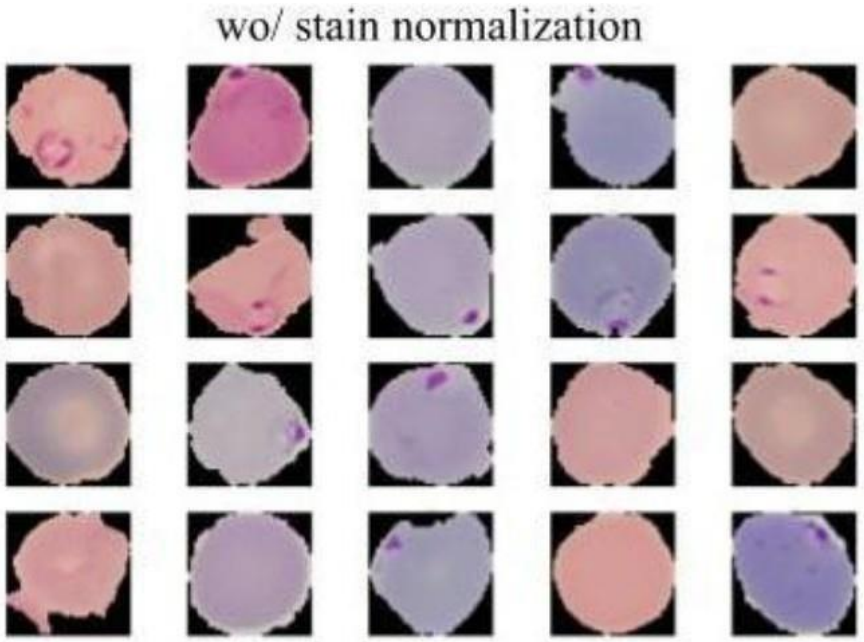


Figure 6.5: Cells without Stain Normalization

for a wide range of computer vision tasks to aid in improved performance, as compared to state-of-the-art methods. We have used transfer learning on the VGG-19 model and the same for ResNet-50 and evaluating their performances which are discussed in further sections.

6.3.2 VGG-19

VGG-19 is a Convolutional Neural Network that is 19 layers deep. VGG-19 is one of a variant of the VGG model. There are other variants of VGG like VGG-11, VGG- 16, and others. VGG was created by Visual Geometry Group at Oxford's and hence thename VGG. It uses deep convolutional neural layers to improve accuracy. VGG-19 Architecture consists of 16 convolutional layers and 3 fully connected layers with five max-pool layers. In this research, we have leveraged transfer learning to train our model on the ImageNet dataset. It is an image database consisting of 14,197,122 images organized according to the WordNet hierarchy. This is an initiative to help researchers, students, and others in the field of image and vision research [43]. In this research, we

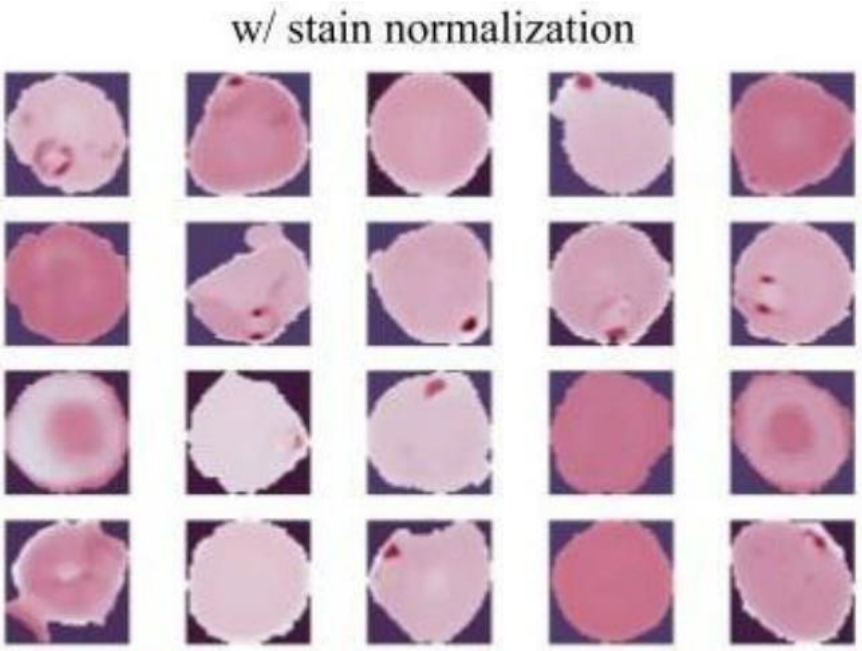


Figure 6.6: Cells with Stain Normalization

have used VGG-19 by leveraging transfer learning with frozen layers, designed model from scratch, and transfer learning with fine-tuning the model on the malaria dataset. The initial model was designed from scratch with all 19 trainable layers on the malaria dataset. The Rectified Linear Units (ReLU) is used as an activation function for the network. The model was compiled with Adam's optimization with binary cross entropy as its loss function [44]. The second model was frozen pre-trained layers. We leveraged transfer learning to get the trained model and used it to get the output of our images. We used it as a simple feature extractor by freezing all the five convolution blocks to make sure their weights don't get updated after each epoch. We used the sigmoid activation function for this model.

The third model was a Fine-tuned model in which we froze the initial three blocks using transfer learning from the ImageNet dataset and made block four and block five trainable on the malaria dataset. We applied fine-tuning to the VGG model, where we unfroze the last two blocks (Block four and Block five) so that their weights get updated in each iteration (per batch of data) as we train our own model. We also applied all the

preprocessing techniques like standardization, data augmentation, and normalization on this model. Sigmoid activation function was used with 2 ways to get the output as1 for malaria or 0 for healthy to our classification problem.

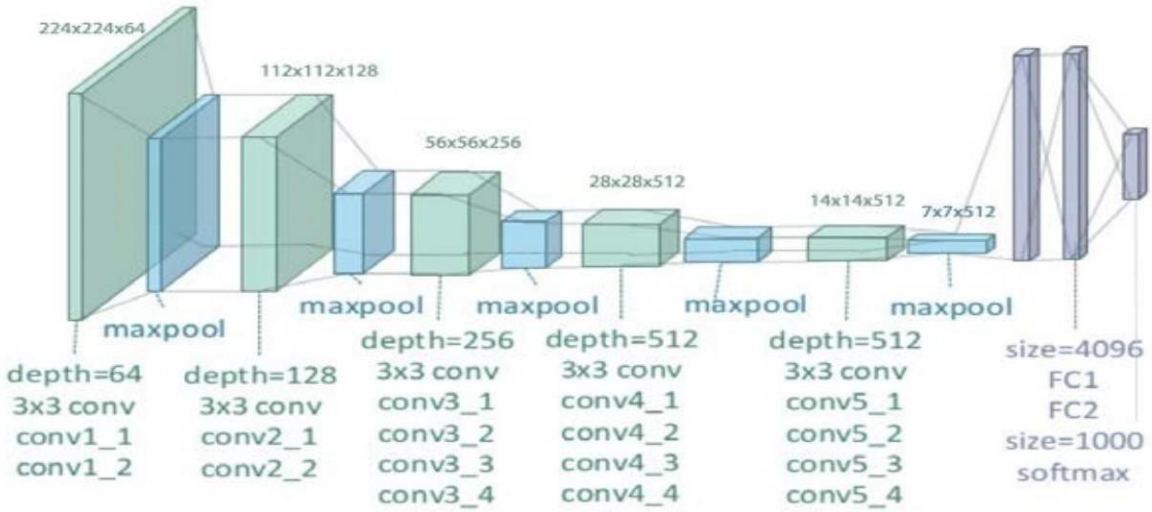


Figure 6.7: VGG-19

The Root Mean Square Prop (RMSProp) is used as a learning rate optimization algorithm with an initial learning rate set as lr = 1e − 4. Root Mean Square Prop (RMSProp) works by keeping an exponentially weighted average of the squares of past gradients. RMSProp then divides the learning rate by this average to speed up convergence.

6.3.3 ResNet-50

Residual Network (ResNet) is a classic neural network, winner of the ImageNet challenge in 2015 [45]. Training very deep neural networks was an issue until ResNet was introduced due to the vanishing gradient problem. ResNet's skip connection technique helped solve the vanishing gradient problem and help to get better results using deep neural networks like ResNet50. In this research, we used the transfer learning with ResNet50 by freezing initial blocks to get the output without updating the weights. We also implemented fine-tuning with preprocessing techniques on the model.

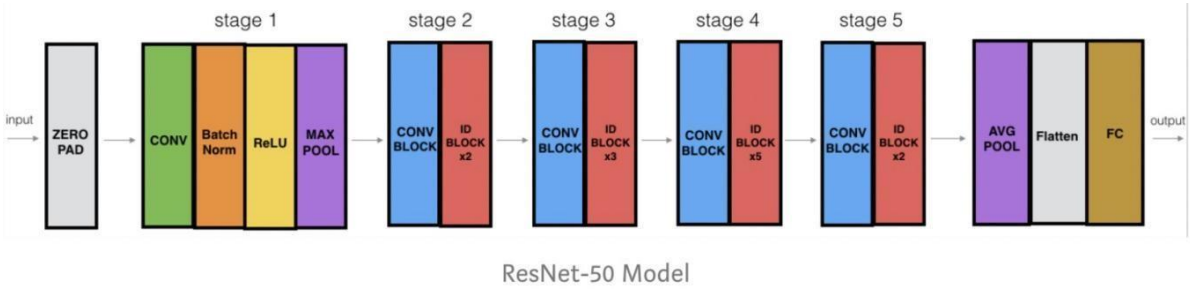


Figure 6.8: ResNet-50

The parameters for the fine tuning model looked:

Total params: 25,686,913

Trainable params: 25,633,793

Non-trainable params: 53,120

The ResNet-50 model consists of 5 stages, each with a convolution block and identity block. Each convolution block has three convolution layers, and each identity block also has three convolution layers. The ResNet-50 has over 25 million trainable parameters. The reason to use sigmoid activation other than softmax is that the task at hand isa binary classification of images and sigmoid performs well with binary classification. Softmax is suitable for multi-class classification [46]. The metrics used to measure the dataset were Accuracy and Loss. These metrics were measured for both training and validation data. We are using the RMSProp optimizer which stands for Root Mean Square. Entropy is measured using binary cross-entropy in batches of size 64.

6.3.4 Ensemble Model

Ensemble learning is a method that is considered to maximize model performance. In this study, we have used ensemble learning. Ensemble learning combines the prediction from multiple models to provide a more robust and accurate network than individual networks. In this research, the idea of ensemble learning is adapted from previous works of literature cited. The process results in ensemble learning that leads to predictions that are better than any individual model [19]. Dietterich et al. [47] used a model

averaging ensemble to achieve state-of-the-art performance in the ImageNet Large Scale Visual Recognition Competition (ILSVRC) 2012 classification task. Figure 5.9 depicts our ensemble-based approach. The predicted output from each network VGG-19 and ResNet-50 is concatenated and averaged to get the final prediction. The Table 5.9 shows the result from the individual networks and the ensemble network.

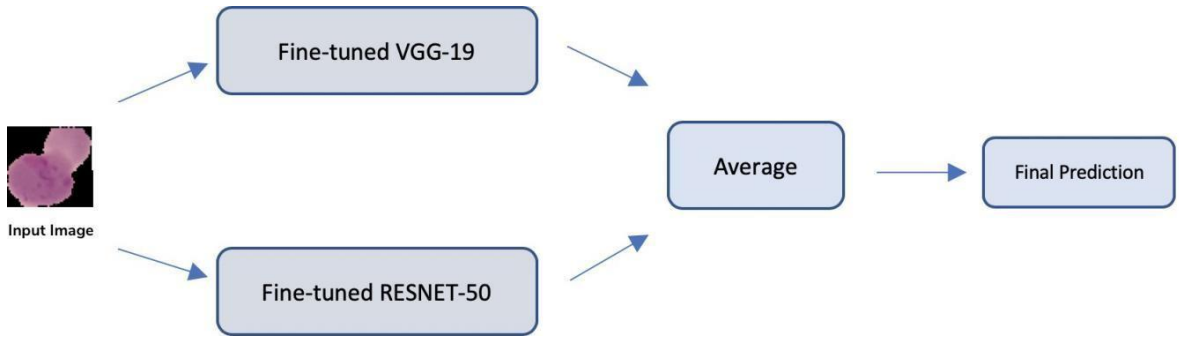


Figure 6.9: Ensemble Model

6.4 Confusion Matrix Analysis

Confusion matrix marks the prediction as true-positive(TP), true-negative(TN), false-positive(FP) and false-negative(FN). Matrix helps to analyze the number of positive and negative prediction. As we can see in the figures, VGG-19 model predicted 276 images wrong out of a total of 8268 samples. Out of 276 misclassified images, 195 are false-positive and 81 were false negative. Similarly in ResNet-50 prediction, we found 142 and 95 false positive and false negative, respectively.

True Positive: when the model correctly classifies positive class the healthy images.

True Negative: when the model correctly classifies negative class the infected images.

False positive: is the case when you classify the infected images as healthy. This is mostly when malaria is in the early stages and it is not easy to spot enough marks to classify as infected.

False negative is the case when the model classifies a healthy cell as infected. This is

usually the case when the model fails to differentiate between red blood cells and mark them as infected.

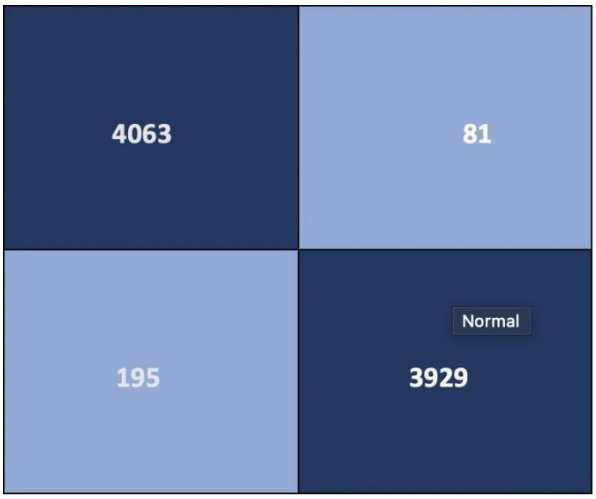


Figure 6.10: Confusion Matrix, VGG-19

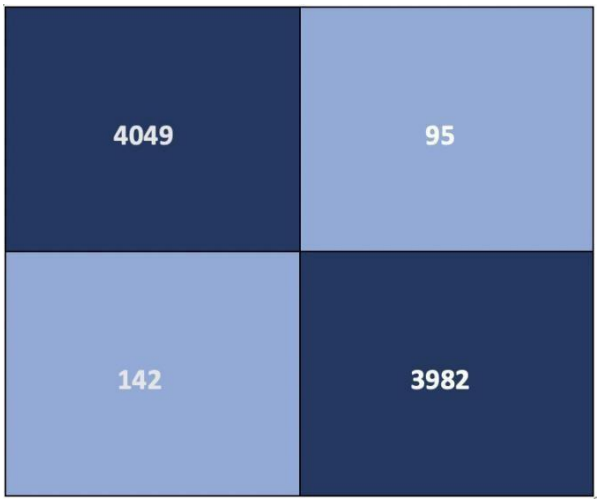


Figure 6.11: Confusion Matrix, ResNet-50

6.5 Performance Parameters

Model performance is evaluated on four different parameters: accuracy, precision, recall, and F1-score.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{5.1}$$

$$Precision = \frac{TP}{TP + FP} \tag{5.2}$$

$$Recall = \frac{TP}{TP + FN} \tag{5.3}$$

$$F1\text{-}score = \frac{2 \times PRECISION \times RECALL}{PRECISION + RECALL} \tag{5.4}$$

SYSTEM TESTING

7. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

7.1 TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.
Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.

- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

5 TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit

testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements

document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

IMPLEMENTATION

8. IMPLEMENTATION

Code for Extracting the Data

The python source used in the study is as presented in this section. This source code was used to extract data from the Ethereum network. Various pythoin libraries were used. They include NumPy, SciPy, Pandas, Matplotlib, network, pickle, SYS and OS.

```
import numpy as np
import  sys
import os
import glob

base_dir = os.path.join('/content/Malaria_dataset/Malaria_dataset')
infected_dir  =  os.path.join(base_dir,'Parasitized')
healthy_dir  =  os.path.join(base_dir,'Uninfected')

infected_files = glob.glob(infected_dir+'/*.png')
healthy_files = glob.glob(healthy_dir+'/*.png')
len(infected_files),  len(healthy_files)
%   \end{lstlisting}
```

```
% \begin{lstlisting}
import numpy as np
import pandas as pd

np.random.seed(42)

files_df = pd.DataFrame({
   'filename': infected_files + healthy_files,
   'label': ['malaria']*len(infected_files) + ['healthy']*len(healthy_files)
}).sample(frac=1, random_state=42).reset_index(drop=True)

files_df.head()
from sklearn.model_selection import train_test_splitfrom
collections import Cou
```

Pre-processing steps

```
train_files, test_files, train_labels, test_labels =
train_test_split(files_df['filename'].values,
files_df['label'].values,
test_size=0.3,
random_state=42)

train_files, val_files, train_labels, val_labels
                                    = train_test_split(train_files,
                                                       train_labels,
                                                       test_size=0.1,
                                                       random_state=42)

print(train_files.shape, val_files.shape, test_files.shape)
print('Train:',Counter(train_labels),'\nVal:',
Counter(val_labels),'\nTest:', Counter(test_labels))
```

```
BATCH_SIZE = 64
NUM_CLASSES = 2
EPOCHS = 25
INPUT_SHAPE = (125, 125, 3)

train_imgs_scaled = train_data / 255.
val_imgs_scaled = val_data / 255.

# encode text category labels
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
le.fit(train_labels)
train_labels_enc = le.transform(train_labels)
val_labels_enc = le.transform(val_labels)

print(train_labels[:6], train_labels_enc[:6])


import tensorflow as tf

# Load the TensorBoard notebook extension (optional)
%load_ext tensorboard

tf.random.set_seed(42)tf._
version_____
```

\chapter{Model 1 CNN from Scratch}#

Model 1: CNN from Scratch

```
inp = tf.keras.layers.Input(shape=INPUT_SHAPE)

conv1 = tf.keras.layers.Conv2D(32, kernel_size=(3, 3),
                                activation='relu', padding='same')(inp)pool1
= tf.keras.layers.MaxPooling2D(pool_size=(2, 2))(conv1)
conv2 = tf.keras.layers.Conv2D(64, kernel_size=(3, 3),
                                activation='relu', padding='same')(pool1)
pool2 = tf.keras.layers.MaxPooling2D(pool_size=(2, 2))(conv2)
conv3 = tf.keras.layers.Conv2D(128, kernel_size=(3, 3),
                                activation='relu', padding='same')(pool2)
pool3 = tf.keras.layers.MaxPooling2D(pool_size=(2, 2))(conv3)

flat = tf.keras.layers.Flatten()(pool3)

hidden1 = tf.keras.layers.Dense(512, activation='relu')(flat)drop1 =
tf.keras.layers.Dropout(rate=0.3)(hidden1)
hidden2 = tf.keras.layers.Dense(512, activation='relu')(drop1)drop2 =
tf.keras.layers.Dropout(rate=0.3)(hidden2)

out = tf.keras.layers.Dense(1, activation='sigmoid')(drop2)

model = tf.keras.Model(inputs=inp, outputs=out)
model.compile(optimizer='adam',
              loss='binary_crossentropy',
```

```
                    metrics=['accuracy'])
model.summary()

callbacks = [reduce_lr, tensorboard_callback]

history = model.fit(x=train_imgs_scaled,
                    y=train_labels_enc,
                    batch_size=BATCH_SIZE,
                    epochs=EPOCHS,
                    validation_data=(val_imgs_scaled, val_labels_enc),
                    callbacks=callbacks,
                    verbose=1)



f, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4)) t =
f.suptitle('Basic CNN Performance', fontsize=12)
f.subplots_adjust(top=0.85, wspace=0.3)

max_epoch = len(history.history['accuracy'])+1
epoch_list = list(range(1,max_epoch))
ax1.plot(epoch_list, history.history['accuracy'], label='Train Accuracy') ax1.plot(epoch_list,
history.history['val_accuracy'], label='Validation Accuracy')ax1.set_xticks(np.arange(1,
max_epoch, 5))
ax1.set_ylabel('Accuracy Value')
ax1.set_xlabel('Epoch')
ax1.set_title('Accuracy')
```

```
l1 = ax1.legend(loc="best")

ax2.plot(epoch_list, history.history['loss'], label='Train Loss') ax2.plot(epoch_list,
history.history['val_loss'], label='Validation Loss')ax2.set_xticks(np.arange(1,
max_epoch, 5))
ax2.set_ylabel('Loss Value')
ax2.set_xlabel('Epoch')
ax2.set_title('Loss')
l2 = ax2.legend(loc="best"
```

Frozen Pre-trained CNN

Transfer Learning Model 2: Frozen Pre-trained CNN

```
vgg = tf.keras.applications.vgg19.VGG19(include_top=False,
                                        weights='imagenet',
                                        input_shape=INPUT_SHAPE)
vgg.trainable = False#
Freeze the layers
for layer in vgg.layers:
    layer.trainable = False

base_vgg = vgg
base_out = base_vgg.output
pool_out = tf.keras.layers.Flatten()(base_out)
hidden1 = tf.keras.layers.Dense(512, activation='relu')(pool_out)drop1 =
tf.keras.layers.Dropout(rate=0.3)(hidden1)
hidden2 = tf.keras.layers.Dense(512, activation='relu')(drop1)drop2 =
tf.keras.layers.Dropout(rate=0.3)(hidden2)
```

```
out = tf.keras.layers.Dense(1, activation='sigmoid')(drop2)

model = tf.keras.Model(inputs=base_vgg.input, outputs=out)
model.compile(optimizer=tf.keras.optimizers.RMSprop(lr=1e-4),
              loss='binary_crossentropy',
              metrics=['accuracy'])
model.summary()

callbacks = [reduce_lr, tensorboard_callback]

history = model.fit(x=train_imgs_scaled,
                    y=train_labels_enc,
                    batch_size=BATCH_SIZE,
                    epochs=EPOCHS,
                    validation_data=(val_imgs_scaled, val_labels_enc),
                    callbacks=callbacks,
                    verbose=1)
model.save('vgg_frozen.h5')
```

Code for Extracting the Data

Pre-processing

```
train_datagen =
tf.keras.preprocessing.image.ImageDataGenerator(
rescale=1./255,
zoom_range=0.05,
rotation_range=25,
width_shift_range=0.05,
height_shift_range=0.05,
shear_range=0.05,
horizontal_flip=True,
fill_mode='nearest')

val_datagen =
tf.keras.preprocessing.image.ImageDataGenerator(rescale=1./255) img_id = 0
sample_generator = train_datagen.flow(
train_data[img_id:img_id+1],
```

```
 train_labels[img_id:img_id+1],
 batch_size=1)

sample = [next(sample_generator) for i in range(0,5)]
fig, ax = plt.subplots(1,5, figsize=(16, 6))
print('Labels:', [item[1][0] for item in sample])
l = [ax[i].imshow(sample[i][0][0]) for i in range(0,5)]

train_generator = train_datagen.flow(train_data,
                                     train_labels_enc,
                                     batch_size=BATCH_SIZE,
                                     shuffle=True)
val_generator = val_datagen.flow(val_data,
                                 val_labels_enc,
                                 batch_size=BATCH_SIZE,
                                 shuffle=False)


\chapter{Model 3 Fine Tuned CNN}

# Transfer Learning
# Model 3: Fine-tuned Pre-trained CNN with Image Augmentation

vgg = tf.keras.applications.vgg19.VGG19(include_top=False,
                                        weights='imagenet',
                                        input_shape=INPUT_SHAPE)
# Freeze the layers
```

```
vgg.trainable = True

set_trainable = False
for layer in vgg.layers:
    if layer.name in ['block5_conv1', 'block4_conv1']:
        set_trainable = True
    if set_trainable:
        layer.trainable = True
    else:
        layer.trainable = False

base_vgg = vgg
base_out = base_vgg.output
pool_out = tf.keras.layers.Flatten()(base_out)
hidden1 = tf.keras.layers.Dense(512, activation='relu')(pool_out)
drop1 = tf.keras.layers.Dropout(rate=0.3)(hidden1)
hidden2 = tf.keras.layers.Dense(512, activation='relu')(drop1)
drop2 = tf.keras.layers.Dropout(rate=0.3)(hidden2)

out = tf.keras.layers.Dense(1, activation='sigmoid')(drop2)

model = tf.keras.Model(inputs=base_vgg.input, outputs=out)
model.compile(optimizer=tf.keras.optimizers.RMSprop(lr=1e-5),
              loss='binary_crossentropy',
              metrics=['accuracy'])

print("Total Layers:", len(model.layers))
```

```
print("Total trainable layers:", sum([1 for l in model.layers if l.trainable]))

callbacks = [reduce_lr, tensorboard_callback]
train_steps_per_epoch = train_generator.n // train_generator.batch_size
val_steps_per_epoch =
val_generator.n // val_generator.batch_size
history = model.fit_generator(train_generator,
                      steps_per_epoch=train_steps_per_epoch,
                     epochs=EPOCHS,
                     validation_data=val_generator,
validation_steps=
val_steps_per_epoch,
verbose=1)
model.save('vgg_finetuned.h5')
```

RESULT

9.RESULT

Research is done on human red blood cell smear images to classify as infected or healthy.The research uses different pre-trained Convolutional Neural Networks with transfer learning and fine-tuning them on the malaria dataset and recording their performances. Research reveals that using different preprocessing techniques like normalization, stain standardization, and scaling does not affect the model performance a whole lot on VGG-19 and ResNet-50, whereas the data augmentation technique has shown promising results on the same. Further, we compared the performance of VGG-19 and ResNet-50 to decide the best performing model. ResNet- 50 with data augmentation achieves anaccuracy of 97.09%.

The basic VGG-19 model achieves an accuracy of 95% whereas was able to achieve 96.24% accuracy after fine-tuning the model and applying the data augmentation technique to the training dataset. Research shows that transfer learning is a great technique that can be used to achieve promising results. Same with ResNet after fine-tuning and applying data augmentation, we were able to achieve the highest accuracy of our experiments of 97.09%.

Earlier state of the art has shown that ResNet best accuracy performance to be 95.7% and have shown ensemble model performs very well with the accuracy of 96.76%

```
Model Performance metrics:
------------------------------
Accuracy: 0.9516
Precision: 0.9517
Recall: 0.9516
F1 Score: 0.9516

Model Classification report:
------------------------------
              precision    recall  f1-score   support

     healthy       0.95      0.96      0.95      4144
     malaria       0.96      0.95      0.95      4124

    accuracy                           0.95      8268
   macro avg       0.95      0.95      0.95      8268
weighted avg       0.95      0.95      0.95      8268


Prediction Confusion Matrix:
------------------------------
                Predicted:
                   healthy malaria
Actual: healthy       3964     180
        malaria        220    3904
```

Figure 9.1: VGG-19 basic CNN model

```
Model Performance metrics:
------------------------------
Accuracy: 0.9321
Precision: 0.9322
Recall: 0.9321
F1 Score: 0.9321

Model Classification report:
------------------------------
              precision    recall  f1-score   support

     healthy       0.93      0.93      0.93      4144
     malaria       0.93      0.93      0.93      4124

    accuracy                           0.93      8268
   macro avg       0.93      0.93      0.93      8268
weighted avg       0.93      0.93      0.93      8268


Prediction Confusion Matrix:
------------------------------
                Predicted:
                   healthy malaria
Actual: healthy       3857     287
        malaria        274    3850
```

Figure 9.2: VGG-19 frozen model

```
Model Performance metrics:
------------------------------
Accuracy: 0.9624
Precision: 0.9629
Recall: 0.9624
F1 Score: 0.9624

Model Classification report:
------------------------------
              precision    recall  f1-score   support

     healthy       0.95      0.98      0.96      4144
     malaria       0.98      0.95      0.96      4124

    accuracy                           0.96      8268
   macro avg       0.96      0.96      0.96      8268
weighted avg       0.96      0.96      0.96      8268


Prediction Confusion Matrix:
------------------------------
                 Predicted:
                    healthy malaria
Actual: healthy        4059      85
        malaria         226    3898
```

Figure 9.3: VGG-19 fine tuned model

which our ResNet model was able to challenge and beat by achieving the accuracy of 97.09%.

Figure 6.4, Figure 6.5, and Figure 6.6 shows the accuracy achieved by the VGG-19 on different settings on the training dataset and validation dataset per epochs. The graphs also compare the binary cross-entropy loss as a function of loss value and epochson the train and validation set.

The Figure 6.7 shows the accuracy achieved by ResNet-50 on the training dataset and validation set per epochs. The graph also shows the loss as a function of loss value and epochs on the train and validation set.

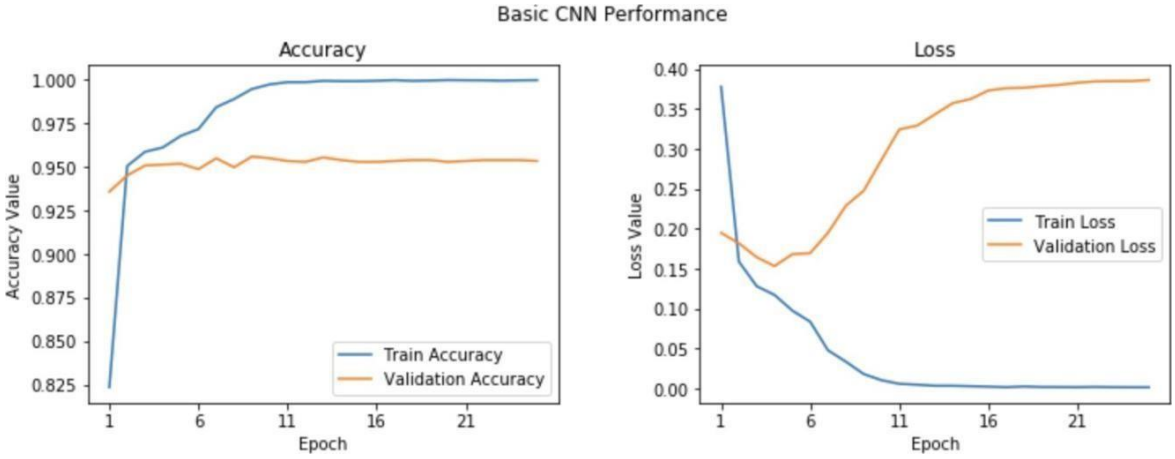


Figure 9.4: Accuracy plot- VGG-19 basic CNN model

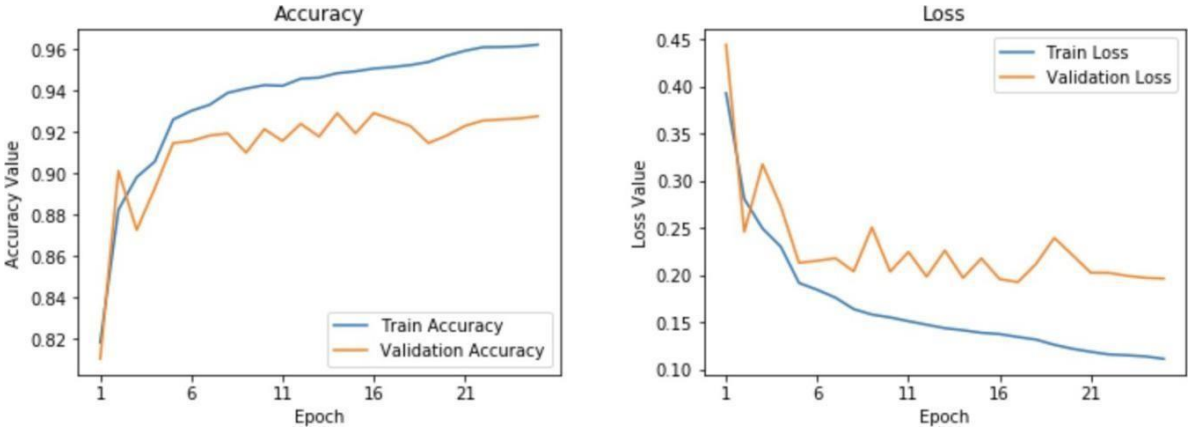


Figure 9.5: Accuracy plot- VGG-19 frozen model

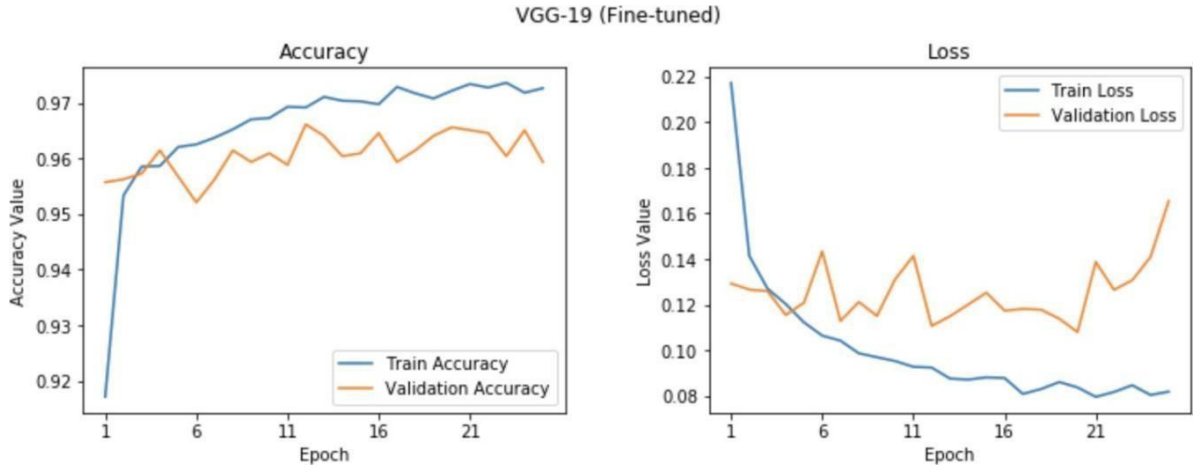


Figure 9.6: Accuracy plot- VGG-19 fine tuned model

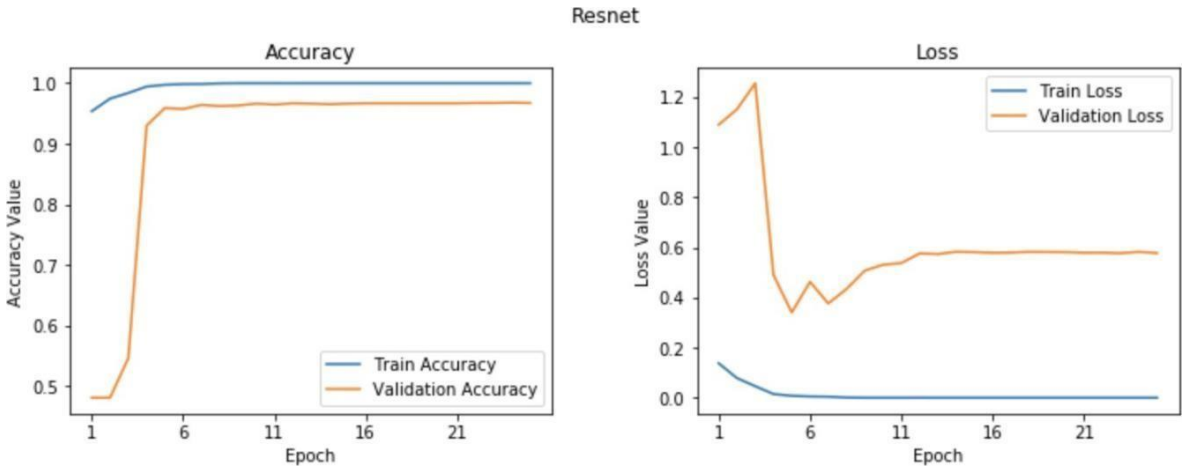


Figure 9.7: Accuracy plot ResNet-50

Model	Accuracy (Test)	Precision	Recall	F1 score
VGG-19 basic	93.21	.9322	.9321	.9321
VGG-19 fine tuned	96.24	.9629	.9624	.9624
ResNet-50	97.09	.9709	.9709	.9709
State of art customized [Rajaraman2018]	94.00	.951	.931	.941
State of art ResNet-50 [Rajaraman2018]	95.70	.969	.945	.957

Figure 9.8: Model performance

DISCUSSION

10. DISCUSSION

We experimented with several preprocessing and post-processing techniques to analyze their effects on the malaria dataset. Preprocessing techniques such as stain normalization and standardization did not improve model performance. These results suggest that features do not have to be on the same scale for the models to be trained.

Data Augmentation technique performed significantly better on the dataset. The result shows that the data augmentation method contributed to increasing model performance. Dropout technique was not implemented in the research to avoid loss of information from images. Test Time augmentation was also a successful approach that increased model performance.

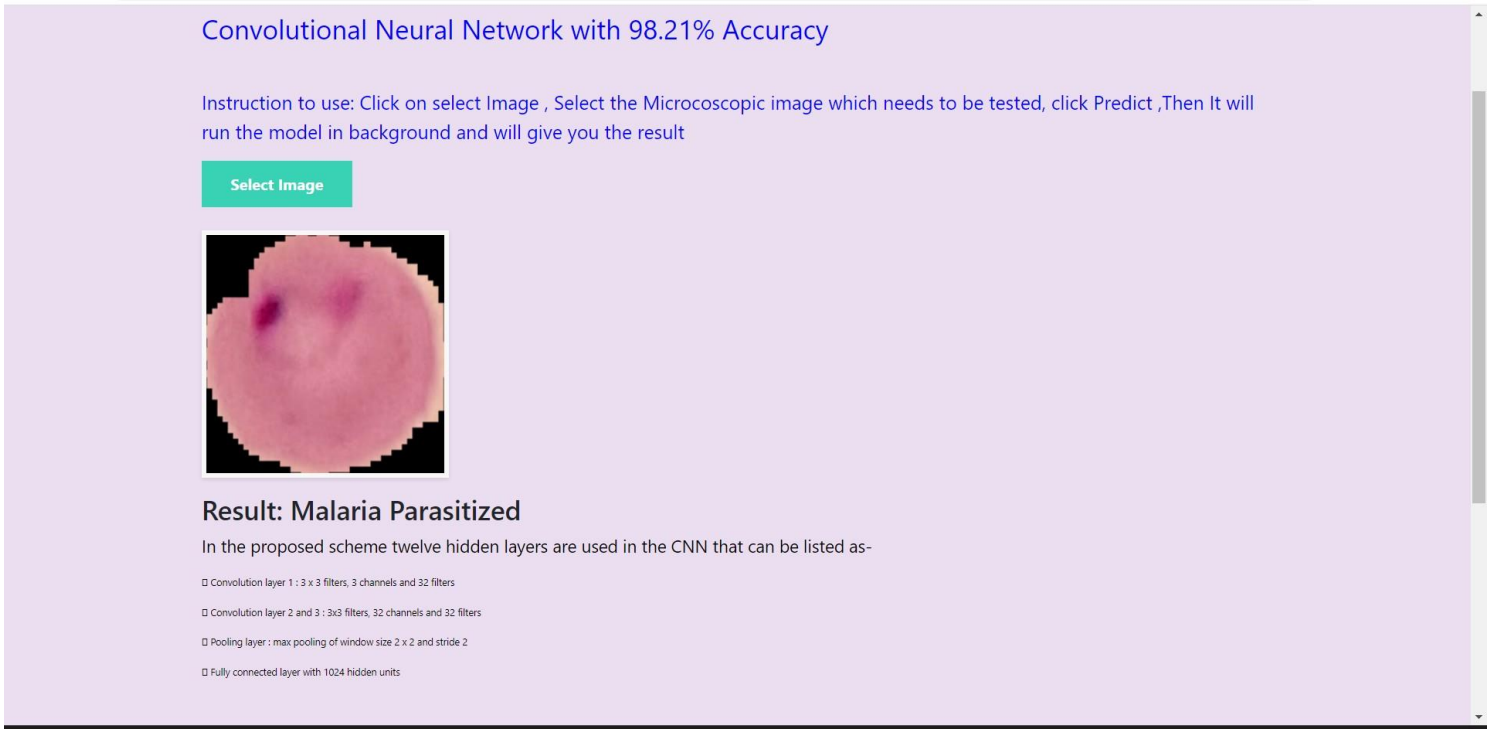
Stain normalization, a preprocessing technique, did not perform well on the malaria dataset. This might be because the methods applied for stain normalization in the research were developed for H&E stains; the samples from the NIH malaria dataset on the contrary do not have enough Hematoxylin and hence the technique was not useful here. We tried preserving the semantic meaning of the image resulted in better performance of the models without stain normalization.

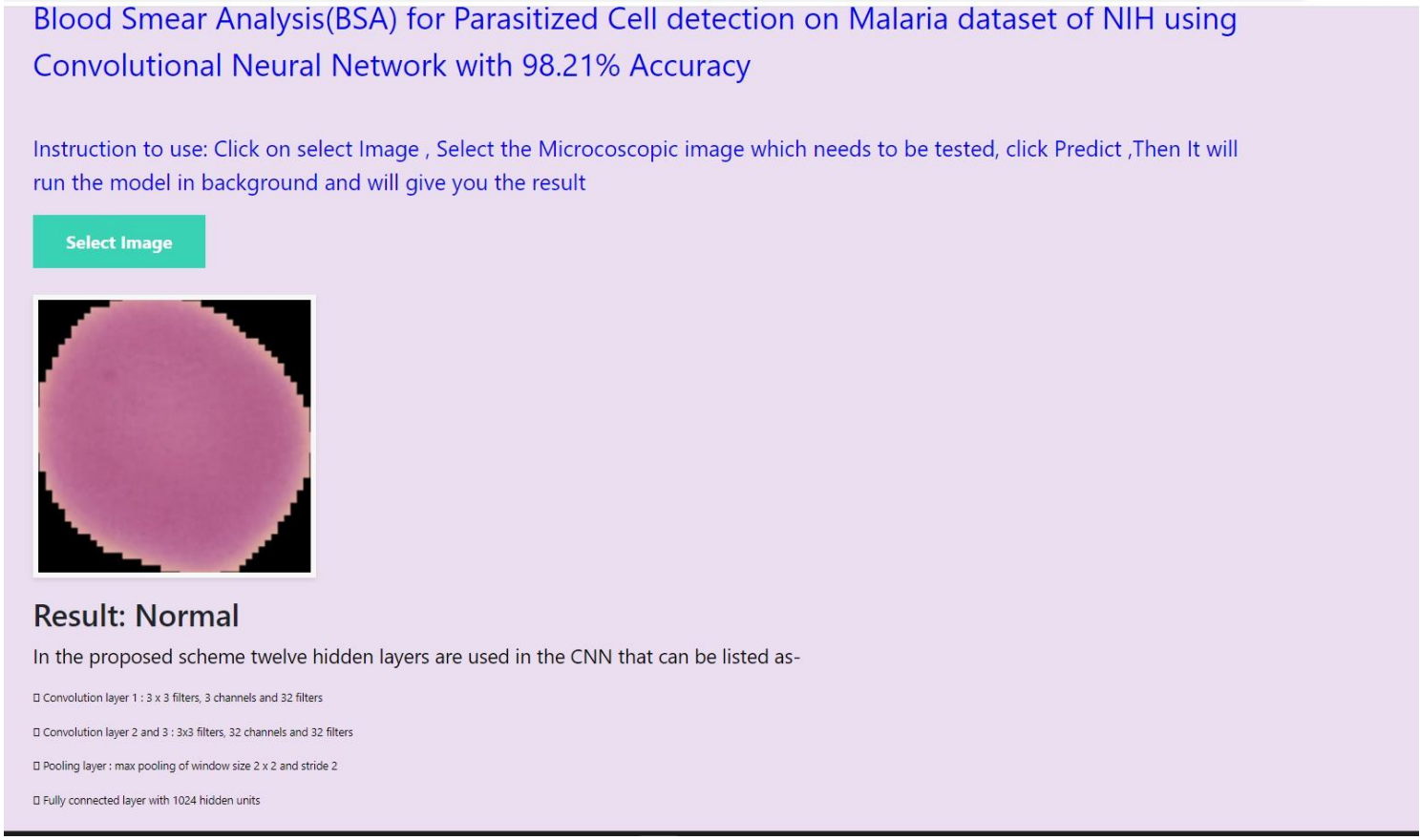
Post-processing techniques, such as the Ensemble model showed good results. In order to check whether these models are biased to a specific test, five different inde-

pendent test sets were evaluated on five different models. Hence, our best performing model does not have a larger variance from the validation sets.

SCREEN SHOTS

11. Screen shots





CONCLUSION

12. CONCLUSION

In this research, we experimented with end-to-end deep learning neural networks to improve malaria diagnosis classification performance. We showed that preprocessing techniques like normalization, standardization, and staining does not contribute muchin improving the performance. Instead, methods like data augmentation showed posi- tive resultsby increasing the performance of the models. We also compared different models such as VGG-19 and ResNet-50 and their performances. We established VGG- 19 and ResNet-50 models from scratch and using transfer learning and hyper tuningthe parameters. Transfer learning is a great technique and could be used to gain satis-factory performance compared to machine learning models that require a lot of feature scaling and engineering. ResNet-50 outperformed all the other models by achieving an accuracy of 97.09%. In future works, we plan to focus on network architecture of themodels used in the research to find the driving performance factors. We plan on findingways to improve the performance by manipulating the network architecture and hyper tuning the features to achieve better performing model

13. Bibliography

[1] Sumit Saha. A Comprehensive Guide to Convolu-tionalNeural Networks. https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53, 2018.

[2] Andrej Trampuz, Matjaz Jereb, Igor Muzlovic, and Rajesh M. Prabhu. Clinical review: Severe malaria. *Critical Care*, 7(4):315–323, aug 2003.

[3] WHO. WHO | This year's World malaria report at a glance. https://www.who.int/malaria/media/world-malaria-report-2018/en/, 2018.

[4] Andrej Karpathy. CS231n Convolutional Neural Networks for Visual Recognition. http://cs231n.github.io/convolutional-networks/, 2018.

[5] Noppadon Tangpukdee, Chatnapa Duangdee, Polrat Wilairatana, and Srivicha Krudsood. Malaria diagnosis: A brief review, 2009.

[6] T. Hänscheid. Current strategies to avoid misdiagnosis of malaria. *Clinical Microbiology and Infection*, 9(6):497–504, jun 2003.

[7] John A. Quinn, Rose Nakasi, Pius K. B. Mugagga, Patrick Byanyima, William Lubega, and Alfred Andama. Deep Convolutional Neural Networks for Microscopy-Based Point of Care Diagnostics. pages 1–12, 2016.

[8] Dev Kumar Das, Madhumala Ghosh, Mallika Pal, Asok K. Maiti, and Chandan Chakraborty. Machine learning approach for automated screening of malaria parasite using light microscopic images. *Micron*, 45:97–106, feb 2013.

[9] Dev Kumar Das, Madhumala Ghosh, Mallika Pal, Asok K. Maiti, and Chandan Chakraborty. Machine learning approach for automated screening of malaria parasite using light microscopic images. *Micron*, 45:97–106, feb 2013.

[10] Han Sang Park, Matthew T. Rinehart, Katelyn A. Walzer, Jen-Tsan Ashley Chi, and Adam Wax. Automated Detection of P. falciparum Using Machine Learn-ing Algorithms with Quantitative Phase Images of Unstained Cells. *PLOS ONE*, 11(9):e0163045, sep 2016.

[11] Wan Azani Mustafa, Ragunathan Santiagoo, Irfan Jamaluddin, Nurul S. Othman, Wan Khairunizam, and M. N.K.H. Rohani. Comparison of Detection Method on Malaria Cell Images. In *2018 International Conference on Computational Approach in Smart Systems Design and Applications, ICASSDA 2018*. Institute of Electrical and Electronics Engineers Inc., sep 2018.

[12] James C. Bezdek, Robert Ehrlich, and William Full. FCM: The fuzzy c-means clustering algorithm. *Computers and Geosciences*, 10(2-3):191–203, jan 1984.

[13] Fang Liu and Rosalind W. Picard. Periodicity, directionality, and randomness: Wold features for image modeling and retrieval. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(7):722–733, 1996.

[14] Kerry. Bradley. US7174215B2 - Method for determining stimulation parameters - Google Patents.

[15] Bartolo, Alexandra, Kenneth P. Camilleri, Philip J. Farrugia and Jonathan C. Borg. Adaptation of Brensen s Thresholding Algorithm for Sketched Line Draw- ings.

[16] Marshall Hargrave. Deep learning definition. https://www.investopedia.com/terms/d/deep-learning.asp, 2019.

[17] Mahdieh Poostchi, Kamolrat Silamut, Richard J. Maude, Stefan Jaeger, and George Thoma. Image analysis and machine learning for detecting malaria. *Trans-lational Research*, 194:36–55, 2018.

[18] Yann Lecun, Yoshua Bengio, and Geoffrey Hinton. Deep learning, may 2015.

[19] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks.

[20] Esra Var and F. Boray Tek. Malaria Parasite Detection with Deep Transfer Learning. In *UBMK 2018 - 3rd International Conference on Computer Science and Engineering*, pages 298–302. Institute of Electrical and Electronics Engineers Inc., dec 2018.

[21] Sivaramakrishnan Rajaraman, Sameer K. Antani, Mahdieh Poostchi, Kamolrat Silamut, Md A. Hossain, Richard J. Maude, Stefan Jaeger, and George R. Thoma. Pre-trained convolutional neural networks as feature extractors toward improved malaria parasite detection in thin blood smear images. *IEEE Transactions on Medical Imaging*, 35(5):315–323, aug 2016.

[22] Shin Hoo-Chang Member, Holger R Roth hoochangshin, Mingchen Gao, Le Lu Senior Member, Ziyue Xu, Isabella Nogues, Jianhua Yao, Daniel Mollura, Ronald M Summers, Hoo-Chang Shin, Holger R Roth, and Le Lu. Deep Convolutional Neural Networks for Computer-Aided Detection: CNN Architectures, Dataset Characteristics and Transfer Learning and Daniel Mollura are with Center for Infectious Disease Imaging HHS Public Access. *IEEE Trans. Med. Imag.*, 35(5):1285–1298, 2016.

[23] Yuhang Dong, Zhuocheng Jiang, Hongda Shen, W. David Pan, Lance A. Williams, Vishnu V.B. Reddy, William H. Benjamin, and Allen W. Bryan. Evaluations of deep convolutional neural networks for automatic identification of malaria infected cells. In *IEEE International Conference on Biomedical and Health Informatics*, pages 101–104. Institute of Electrical and Electronics Engineers Inc., apr 2017.

[24] Zhaohui Liang, Andrew Powell, Ilker Ersoy, Mahdieh Poostchi, Kamolrat Silamut, Kannappan Palaniappan, Peng Guo, Md Amir Hossain, Antani Sameer, Richard James Maude, Jimmy Xiangji Huang, Stefan Jaeger, and George Thoma. CNN-based image analysis for malaria diagnosis. In *Proceedings - 2016 IEEE International Conference on Bioinformatics and Biomedicine, BIBM 2016*, pages 493–496. Institute of Electrical and Electronics Engineers Inc., jan 2017.

[25] Dhanya Bibin, Madhu S. Nair, and P. Punitha. Malaria Parasite Detection from Peripheral Blood Smear Images Using Deep Belief Networks. *IEEE Access*, 5:9099–9108, 2017.

[26] Gopalakrishna Pillai Gopakumar, Murali Swetha, Gorthi Sai Siva, and Gorthi R. K Sai Subrahmanyam. Convolutional neural network-based malaria diagnosis from focus stack of blood smear images acquired using custom-built slide scanner. *Journal of Biophotonics*, 11(3):e201700003, mar 2018.

[27] Martin Drahansky, M.t Paridah, Amin Moradbak, A.Z Mohamed, Folahan abdulwahab taiwo Owolabi, Mustapha Asniza, and Shawkataly H.P Abdul Khalid. Classification of Malaria-Infected Cells Using Deep Convolutional Neural Networks. *Intech*, i(tourism):13, 2016.

[28] Dayong Wang, Aditya Khosla, Rishab Gargeya, Humayun Irshad, and Andrew H. Beck. Deep Learning for Identifying Metastatic Breast Cancer. (September), 2016.

[29] Ed Corbett. Real-World Benefits of Machine Learn-ingin Healthcare. https://www.healthcatalyst.com/ clinical-applications-of-machine-learning-in-healthcare, 2017.

[30] Jaeger S. Malaria Datasets | National Library of Medicine, 2019.

[31] Francesco Ciompi, Oscar Geessink, Babak Ehteshami Bejnordi, Gabriel Silva De Souza, Alexi Baidoshvili, Geert Litjens, Bram Van Ginneken, Iris Nagtegaal, and Jeroen Van Der Laak. The importance of stain normalization in colorectal tissue classification with convolutional networks. Technical report.

[32] Google. Cloud computing services | google cloud. https://cloud.google.com/, 2020.

[33] Eshant Sah. Convolution -in deep learning. convolutional neural net- work(cnn) | by eshant sah | medium. https://medium.com/@eshant.sah/convolution-in-deep-learning-62eb08b42070, 2019.

[34] Andreea Anghel, Milos Stanisavljevic, Sonali Andani, Nikolaos Papandreou, Jan Hendrick Rüschoff, Peter Wild, Maria Gabrani, and Haralampos Pozidis. A High-Performance System for Robust Stain Normalization of Whole-Slide Imagesin Histopathology. *Frontiers in Medicine*, 6, sep 2019.

[35] Adnan Mujahid Khan, Nasir Rajpoot, Darren Treanor, and Derek Magee. A non-linear mapping approach to stain normalization in digital histopathology images using image-specific color deconvolution. *IEEE Transactions on Biomedical Engineering*, 61(6):1729–1738, 2014.

[36] Babak Ehteshami Bejnordi, Geert Litjens, Nadya Timofeeva, Irene Otte-Höller, André Homeyer, Nico Karssemeijer, and Jeroen A.W.M. Van Der Laak. Stain specific standardization of whole-slide histopathological images. *IEEE Transactions on Medical Imaging*, 35(2):404–415, feb 2016.

[37] Neeraj Kumar, Ruchika Verma, Sanuj Sharma, Surabhi Bhargava, Abhishek Vahadane, and Amit Sethi. A Dataset and a Technique for Generalized Nuclear Segmentation for Computational Pathology. *IEEE Transactions on Medical Imaging*, 36(7):1550–1560, jul 2017.

[38] Ubydul Haque, Syed Masud Ahmed, Shahed Hossain, Mamun Huda, Awlad Hossain, Mohammad Shafiul Alam, Dinesh Mondal, Wasif Ali Khan, Mohammod Khalequzzaman, and Rashidul Haque. Malaria Prevalence in Endemic Districts of Bangladesh. *PLoS ONE*, 4(8):e6737, aug 2009.

[39] Robert A. van den Berg, Huub C.J. Hoefsloot, Johan A. Westerhuis, Age K. Smilde, and Mariët J. van der Werf. Centering, scaling, and transformations: Improving the biological information content of metabolomics data. *BMC Genomics*, 7:142, jun 2006.

[40] Teresa Araú jo, Guilherme Aresta, Eduardo Castro, José Rouco, Paulo Aguiar, Catarina Eloy, Antó nio Poló nia, and Auré lio Campilho. Classification of breast cancer histology images using Convolutional Neural Networks. 2017.

[41] Maayan Frid-Adar, Eyal Klang, Michal Amitai, Jacob Goldberger, and Hayit Greenspan. Synthetic Data Augmentation using GAN for Improved Liver Lesion Classification. Technical report.

[42] Ali Sharif Razavian, Hossein Azizpour, Josephine Sullivan, and Stefan Carlsson. CNN features off-the-shelf: An astounding baseline for recognition. In *IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, pages 512–519. IEEE Computer Society, sep 2014.

[43] Princeton University Stanford Vision Lab, Stanford University. ImageNet.

[44] Diederik P. Kingma and Jimmy Lei Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations, ICLR 2015* -

Conference Track Proceedings. International Conference on Learning Representations, ICLR, dec 2015.

[45] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, volume 2016-Decem, pages 770–778. IEEE Computer Society, dec 2016.

[46] A. c Bharadwaj Reddy and D. Sujitha Juliet. Transfer learning with RESNET-50 for malaria cell-image classification. In *Proceedings of the 2019 IEEE International Conference on Communication and Signal Processing, ICCSP 2019*, pages 945–949. Institute of Electrical and Electronics Engineers Inc., apr 2019.

[47] Thomas G. Dietterich. Ensemble methods in machine learning. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, volume 1857 LNCS, pages 1–15. Springer Verlag, 2000.